

Priyanka — Interview Recovery Plan

Why this exists: You attended Infosys, Altimetrik, and Photon and didn't clear. Altimetrik's feedback was *"project understanding and coding not strong."* This plan fixes **exactly those two things** — nothing else. It is a *practice* plan, not a reading plan.

Made: 8 Jul 2026 · **Sprint:** 10 focused days · then permanent daily habits until you have an offer.

PART 1 — The honest diagnosis (read this once, believe it)

You are **not failing because you don't know enough**. I read every question all three companies asked. They are all standard mid-level questions, and you already have written answers for almost every one in your packs. You are failing on two specific execution skills:

1. **Talking about your own projects with confidence and depth.** Altimetrik asked *"In the Angular migration you mentioned, what challenges did you face?"* — that's about YOUR resume. Photon asked *"Is your app microservices? How many services?"* — again YOUR project. When you hesitate on these, the interviewer stops trusting the whole resume. This is the #1 reason you're getting rejected.
2. **Writing code live, from a blank screen, while someone watches.** Stream API highest-salary-by-department, first non-repeating character, a form with a disabled submit button. You *know* these. But reading a solution $\neq$ producing it under pressure. This is a muscle. It only grows by typing, not reading.

The cure for both is the same: speak out loud and type from scratch, every single day. Passive re-reading of your packs will NOT move the needle. You've already read them. Now you perform them.

PART 2 — The pattern (this is your unfair advantage)

All three companies asked the **same handful of topics**. If you master this short list, you cover ~80% of every interview you'll face:

Rank	Topic	Asked by	Your source file
1	Angular lifecycle hooks	Infosys, Altimetrik, Photon (ALL 3)	priyanka.../04-angular.md
2	Component communication	ALL 3	04-angular.md
3	Observable / Subject / BehaviorSubject / RxJS	Altimetrik, Photon	04-angular.md , robogebra.../01-frontend.md
4	Unsubscribe / memory leaks / takeUntil	Photon (heavily)	robogebra.../01-frontend.md
5		Altimetrik, Photon	robogebra.../01-frontend.md

Rank	Topic	Asked by	Your source file
	Restrict multiple API clicks (debounce + switchMap)		
6	Reactive vs template forms + disabled submit	Altimetrik, Photon	04-angular.md
7	Route guards / lazy loading / resolver / routing design	Photon	04-angular.md
8	Global exception handling (front & back)	Altimetrik, Photon	robogebra.../08-error-handling.md
9	Java Streams (groupingBy/maxBy) + HashMap internals + immutable class	Altimetrik	priyanka.../05-java.md , 12-java17-features.md
10	Interface vs abstract, try/catch/finally	Photon	robogebra.../03-java.md
11	Your microservices architecture	Photon	robogebra.../06-microservices.md

Notice: Angular + RxJS is more than half of everything. That's your strongest area on paper. Own it and you win most of the interview.

PART 3 — The daily non-negotiables (do these EVERY day, all 10 days, ~30 min)

These are habits, not topics. They run in the background of the whole sprint:

-  **Project story out loud (10 min):** Pick ONE project (RoboGebra / EasyVisa / the Angular migration). Say your 60-second pitch, then answer "*what was the hardest challenge and how did you solve it?*" out loud, standing up, no notes. Record yourself on your phone. Listen back once. Are you confident or hedging?
-  **One coding problem typed from BLANK (15 min):** Open a blank file. Solve one problem with **zero peeking**. Only after you finish (or get stuck for 10 min) do you check the answer. Typing beats reading 10x.
-  **5 flashcards out loud (5 min):** Spaced repetition on the previous day's topic.

If you do *only* these habits and nothing else, you will still interview dramatically better than last time.

PART 4 — The 10-day sprint

Front-loaded: highest-frequency topics first, so if an interview pops up on day 4 you've already covered the most-asked material. Each day = **the daily habits above + the focus block below (~1.5–2 hrs)**.

Days 1–4: Angular + RxJS (your bread and butter — half of every interview)

Day 1 — Lifecycle + Component communication (the two most-asked topics on Earth) - Read [04-angular.md](#) sections on lifecycle & communication ONCE, then **close it** and write out from memory: - All

lifecycle hooks in order + one sentence on when each fires. Out loud. - `ngOnChanges` vs `ngDoCheck` (Photon asked this exactly). - Every way components communicate: `@Input` / `@Output`, `@ViewChild`, a shared service with `BehaviorSubject`, routing params. - **Type from blank:** parent→child with `@Input`, child→parent with `@Output` EventEmitter. Make it compile in your head. - **Drill answer:** "How to change text colour on click without using classes?" (Infosys) → `[style.color]` binding or `ElementRef` / `Renderer2`. Say it in 20 seconds.

Day 2 — Observables, Subject vs BehaviorSubject, hot vs cold - From memory, explain: what an Observable is, Observable vs Observer, cold vs hot, `Subject` vs `BehaviorSubject` (`BehaviorSubject` holds a current value + emits it to new subscribers). - **Type from blank:** create an Observable 3 ways — `new Observable`, `of(...)`, `BehaviorSubject`. Subscribe and log. - **Drill:** "Two independent APIs needed in one subscription?" → `forkJoin`. "What if one fails?" → the whole `forkJoin` errors; wrap each inner call in `catchError(() => of(fallback))` so one failure doesn't kill the rest. Say this fluently — Photon asked it.

Day 3 — Memory leaks + unsubscribe + restricting API calls (Photon's favourite) - Explain out loud: what happens if you DON'T unsubscribe → the subscription lives after the component is destroyed → callbacks keep firing → memory leak + duplicate API calls + errors on dead components. - The fix patterns: `async` pipe (auto-unsubscribe), `takeUntil(destroy$)`, `takeUntilDestroyed()` (Angular 16), manual `Subscription.unsubscribe()`. - **Type from blank + say it:** "Button clicked multiple times firing multiple API calls — how do you stop it?" → disable button while in-flight, or `switchMap` (cancels previous), or `exhaustMap` (ignores new clicks until current finishes), or `debounceTime`. Type the `switchMap` version. This exact question came up at BOTH Altimetrik and Photon.

Day 4 — Forms + Routing + Guards + Lazy loading - Reactive vs template-driven (know 3 differences cold). - **Type from blank (Photon's Q34):** a standalone component with a reactive form, username + password, submit button `[disabled]="form.invalid"`. Get it working end to end without looking. - Routing design for `home/admin/login/user/product/productList/productDetail` (Photon's Q14) — sketch the routes on paper. Which guard protects them (`CanActivate`), when you'd use a `Resolver`, lazy loading + its downside (initial navigation delay on first load of the chunk).

Days 5–7: Java core + live coding (your weakest area — where Altimetrik failed you)

Day 5 — Streams (this is THE coding question Altimetrik asked) - **Type from blank, 3 times until fluent:** highest salary per department using Stream API: `java Map<String, Optional<Employee>> topByDept = employees.stream().collect(Collectors.groupingBy(Employee::getDepartment, Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))));` - Also type from blank: first non-repeating character (use `LinkedHashMap<Character, Integer>` counts, then find first with count 1). Altimetrik asked this too. - Know: when was `forEach` introduced (Java 8), stream vs loop.

Day 6 — HashMap internals + immutable class + collections - Explain out loud: how HashMap works internally (hashing → bucket → linked list → tree after 8 collisions in Java 8+), `HashMap` vs `ConcurrentHashMap`, `ArrayList` vs `LinkedList` and which for searching (`ArrayList` — $O(1)$ index access). - **Type from blank:** a fully immutable class (final class, final fields, no setters, defensive copy of mutable fields in constructor & getters). Altimetrik asked the criteria — be able to write it, not just list rules.

Day 7 — OOP live coding (Photon's backend round) - **Type from blank, all of these:** a Java array holding strings and numbers (`Object[]`), an interface, an object of that interface (anonymous class AND

lambda if functional). - Interface vs abstract class with a real-world example you can say in 15 seconds (e.g. "interface = a contract like `PaymentGateway` ; abstract class = shared base like `AbstractRepository` with common code"). - try/catch/finally: what `finally` is for even when try succeeds (cleanup — close resources), what happens if you remove `catch` (exception propagates up / must be declared).

Days 8–9: Spring / backend / your project architecture

Day 8 — Spring Boot essentials + exception handling - `@Controller` vs `@RestController`, `PUT` vs `PATCH` (full replace vs partial update — Altimetrik asked), Spring Boot validation (`@Valid` + `@NotNull` + `@ControllerAdvice`). - Global exception handling front AND back: Angular `HttpInterceptor` for errors + Spring `@ControllerAdvice` / `@ExceptionHandler`. Both companies asked. Read `robogebra.../08-error-handling.md`, then explain the flow from memory. - "If request body is missing, how do you handle the error?" (Altimetrik) → validation returns 400; interceptor catches and shows a toast.

Day 9 — YOUR project, deep (the #1 thing that's rejecting you) - Open `robogebra.../06-microservices.md` and `README.md`. Memorize the FACTS: it's a microservices hybrid — portal (Spring Boot 3.2, Java 17, MongoDB) ↔ CRM (Node/Express) over REST, Cognito auth (3 pools), Angular 16 web + Ionic mobile. - **Say out loud, no notes, as if to an interviewer:** 1. "Is your app microservices? How many services?" → give the real number and name them. 2. "Walk me through the architecture." → draw it on paper while you talk. 3. "What challenge did you face in the Angular migration?" → ONE concrete STAR story: v2/v9 → 16, breaking changes, how you tested repo-by-repo to avoid disruption. 4. One hard challenge from RoboGebra (the debounce + switchMap real-time graph story from `10-projects-deep-dive.md` is perfect — it shows RxJS depth AND project ownership at once). - **Honesty rule:** you're on Angular 16 with `zone.js`, NOT `zoneless/signals`. When asked, say "we're on 16 with `zone.js` and `OnPush` where needed; here's how `zoneless/signals` work and how I'd migrate." Interviewers respect this far more than bluffing.

Day 10: Full mock interview

- Do a complete dry run in order, out loud, timed: 60-sec intro → project deep-dive → 2 live coding problems (typed, no peeking) → Angular/RxJS rapid-fire → Spring → 2 questions for the interviewer.
- Best option: ask a friend to play interviewer using the `Interview_Questions.pdf`. No friend? Record yourself answering each question cold, then watch it back and mark every place you hesitated. Those hesitations ARE your remaining gaps — drill them again.

PART 5 — The interview-day checklist

- ☐ 60-second intro, smooth, no notes
- ☐ Can name & explain all Angular lifecycle hooks and every component-communication method
- ☐ Can explain (and code) unsubscribe patterns + how to stop duplicate API calls with `switchMap`
- ☐ Can write the Stream "highest salary per department" from a blank screen
- ☐ Can write an immutable class and a first-non-repeating-char from blank
- ☐ Can draw your architecture and answer "how many microservices" instantly
- ☐ One rock-solid STAR story per project (migration challenge + RoboGebra real-time graph)

- [] Honest about Angular 16/zone.js reality vs zoneless
 - [] 2–3 thoughtful questions ready for the interviewer
-

PART 6 — The mindset that gets you the offer

- **Interviewers hire confidence in your own work.** When you own your project story, they forgive gaps elsewhere. When you're shaky on your OWN resume, they doubt everything.
- **"I'm not 100% sure, but here's how I'd approach it"** beats silence or bluffing every time. Think out loud during coding — they're grading your reasoning, not just the final answer.
- **You have 5 years and real, impressive projects.** RoboGebra (AI math platform), EasyVisa, four-repo Angular 16 migration — these are genuinely strong. Speak like the engineer who built them, because you are.
- **Three rejections is a pattern you can now fix**, not a verdict. You now know the exact questions and the exact two gaps. Most candidates never get that clarity. Use it.

You've got this, Priyanka. Follow the daily habits religiously — speak out loud, type from blank — and the next one is yours.